ANALITYKA GOSPODARCZA / DATA MINING

Laboratorium 2

1. Dlaczego czyścimy dane etapami

Surowy zbiór może zawierać błędy techniczne i wartości niezgodne z regułami. Dobrze zaplanowany proces można uruchomić ponownie i skontrolować.

```
import numpy as np
import pandas as pd

surowe = pd.read_csv("zamowienia_surowe.csv")
robocze = surowe.copy()
print(surowe.shape)
```

- np to zwyczajowy skrót biblioteki NumPy, przydatnej do obliczeń tablicowych i obsługi np.nan.
- copy() tworzy niezależny obiekt do transformacji; dane surowe pozostają punktem odniesienia.
- Dobry porządek: stan wejścia, duplikaty, teksty, daty, reguły, braki, imputacja, cechy, testy, zapis.

Kiedy używać: Gdy wynik ma być odtwarzalny lub kiedy musisz pokazać wpływ każdej decyzji.

Na co uważać: Nie nadpisuj bez potrzeby jedynej kopii danych źródłowych.

2. Stan początkowy: słownik i liczniki

Słownik zbiera różne miary kontrolne pod czytelnymi nazwami. Pozwala porównać stan przed i po czyszczeniu.

```
stan_startowy = {
    "wiersze": len(surowe),
    "kolumny": surowe.shape[1],
    "unikalne_zamowienia": surowe["zamowienie_id"].nunique(),
    "duplikaty": int(surowe.duplicated().sum()),
}
print(stan_startowy)
```

- len() liczy wiersze, shape[1] liczbę kolumn.
- nunique() liczy różne, niepuste wartości w kolumnie.
- duplicated().sum() daje liczbę dodatkowych kopii pełnych rekordów.
- int() zamienia wynik na zwykłą liczbę całkowitą Pythona.

Kiedy używać: Przed zmianami, aby później wykazać ich liczbę i skutki.

Na co uważać: nunique() domyślnie nie liczy brakujących identyfikatorów. W razie potrzeby użyj dropna=False.

3. Oglądanie i usuwanie duplikatów

Najpierw obejrzyj powtórzenia, później usuń tylko te, których źródło rozumiesz.

```
podejrzane = surowe[surowe.duplicated(keep=False)]
print(podejrzane.sort_values("zamowienie_id"))
robocze = surowe.drop_duplicates().copy()
```

- keep=False oznacza wszystkie kopie identycznego rekordu.

- drop_duplicates() domyślnie pozostawia pierwszą kopię każdego pełnego wiersza.
- Dodatkowe copy() zapewnia niezależny obiekt dla następnych zmian.

Kiedy używać: Gdy dokładnie ten sam rekord trafił do pliku kilka razy.

Na co uważać: Powtórzenie identyfikatora przy innych wartościach nie zostanie usunięte przez domyślne drop_duplicates(). Taki problem wymaga osobnej decyzji.

4. Funkcja czyszcząca i zwracanie dwóch wyników

Zapis procesu w funkcji pozwala go wielokrotnie uruchamiać na danych o tym samym schemacie.

```
def przygotuj_zamowienia(dane: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    wynik = dane.copy()
    historia = []
    # tutaj wykonywane są kolejne etapy czyszczenia
    return wynik, pd.DataFrame(historia, columns=["krok", "liczba", "opis"])

czyste, dziennik = przygotuj_zamowienia(surowe)
```

- dane: pd.DataFrame to adnotacja oczekiwanego typu argumentu.
- -> tuple[...] opisuje oczekiwaną postać wyniku: dwie tabele.
- return wynik, dziennik zwraca parę obiektów, które można przypisać do dwóch nazw.

Kiedy używać: Do powtarzalnego procesu, zamiast serii ręcznych poprawek w komórkach.

Na co uważać: Adnotacja typu nie przeprowadza automatycznie konwersji ani kontroli zawartości. Funkcja musi sama wykonać potrzebne sprawdzenia.

5. Dziennik zmian: list.append() i DataFrame

Zbieraj informacje o decyzjach w trakcie procesu, zamiast odtwarzać je po fakcie.

```
historia = []
przed = len(surowe)
po = len(surowe.drop_duplicates())
historia.append(["deduplikacja", przed - po, "usunięto pełne kopie"])
historia.append(["daty", None, "sprawdzono format dat"])
dziennik = pd.DataFrame(historia, columns=["krok", "liczba", "opis"])
print(dziennik)
```

- Lista historia przechowuje kolejne wiersze dziennika.
- append() dopisuje jeden wiersz w postaci listy.
- None może oznaczać, że liczba nie dotyczy danego kroku.
- DataFrame(..., columns=...) nadaje kolumnom jasne nazwy.

Kiedy używać: Kiedy potrzebny jest ślad audytowy: co zmieniono, w ilu wierszach i dlaczego.

Na co uważać: Nie wystarczy zapisać „naprawiono dane”. Raport powinien umożliwiać ocenę konkretnej transformacji.

6. Wykrywanie kolumn tekstowych: select_dtypes()

Można wykonać tę samą operację czyszczenia na wszystkich kolumnach wybranego typu.

```
tekstowe = robocze.select_dtypes(include="object").columns
for kolumna in tekstowe:
    robocze[kolumna] = robocze[kolumna].astype("string").str.strip()
```

- select_dtypes(include="object") wybiera kolumny o typie object.
- columns zwraca nazwy znalezionych pól.
- astype("string") przełącza na tekstowy typ pandas, a .str.strip() usuwa spacje z początku i końca napisów.

Kiedy używać: Gdy nazwy kategorii mają przypadkowe spacje, które tworzą pozorne osobne grupy.

Na co uważać: object nie obejmuje wszystkich możliwych kolumn tekstowych. W nowszych zbiorach część pól może już mieć typ string. Najbezpieczniej sprawdzić dtypes.

7. Normalizacja kategorii: lower() i map()

Sama zamiana na małe litery nie zawsze wystarczy. Mapowanie nadaje dopuszczalnym wariantom jedną docelową nazwę.

```
mapa_kanalow = {"internet": "Online", "online": "Online",
                "sklep": "Stacjonarny", "salon": "Stacjonarny"}
robocze["kanal"] = (robocze["kanal"]
                    .str.lower()
                    .map(mapa_kanalow))
```

- .str.lower() zamienia litery w każdym niepustym napisie na małe.
- map(słownik) podstawia wartość ze słownika dla każdego elementu Series.
- Wynikowe kategorie są spójne i nadają się do grupowania.

Kiedy używać: Kiedy to samo pojęcie zapisano w różny sposób.

Na co uważać: Wartości nieobecne w słowniku mapowania zmienią się w braki. Zanim użyjesz map(), sprawdź unique() i przygotuj pełną listę wariantów.

8. Daty: pd.to_datetime() i errors="coerce"

Daty zapisane jako tekst trzeba przekonwertować do typu daty, aby można je było porównywać i wyliczać różnice.

```
robocze["data_zakupu"] = pd.to_datetime(
    robocze["data_zakupu"], errors="coerce")
print(robocze["data_zakupu"].dtype)
```

- pd.to_datetime() próbuje rozpoznać format daty.
- errors="coerce" zamienia nierozpoznawalną datę na NaT, odpowiednik braku dla dat.
- Daty po konwersji można np. filtrować po okresie i porównywać chronologicznie.

Kiedy używać: Przed analizą czasu, sezonowości, długości realizacji i audytem chronologii.

Na co uważać: Niepoprawne formaty nie zostaną automatycznie „naprawione”, tylko trafią do braków. Policz NaT po konwersji i w razie potrzeby określ format=... .

9. Reguły jakości: between(), notna(), negacja ~

Reguły biznesowe można zapisać jako maski wierszy, które nie powinny występować.

```
zla_ocena = (robocze["ocena_obslugi"].notna()
             & ~robocze["ocena_obslugi"].between(1, 5))
ujemna_kwota = robocze["wartosc_zamowienia"] < 0
print(zla_ocena.sum(), ujemna_kwota.sum())
```

- between(1, 5) oznacza wartości z przedziału domkniętego, czyli 1 i 5 są dozwolone.
- ~ neguje maskę logiczną, a & oznacza AND element po elemencie.
- notna() pozwala odróżnić nieprawidłową wartość od braku danych.
- sum() zlicza True w masce.

Kiedy używać: Do identyfikowania niepoprawnych wieku, ocen, rabatów, opłat lub liczników.

Na co uważać: Zależnie od typu brakujące wartości mogą mieć inną semantykę porównania. Zdefiniuj jawnie, czy brak sam w sobie jest błędem.

10. Zamiana błędów na braki: loc i np.nan

Jeżeli nie znamy poprawnej wartości, zwykle bezpieczniej oznaczyć ją jako brak niż wymyślać zastępnik.

```
maska = robocze["rabat_proc"].notna() & ~robocze["rabat_proc"].between(0, 100)
robocze.loc[maska, "rabat_proc"] = np.nan
```

- loc[wiersze, kolumna] wybiera wskazane miejsca w DataFrame.
- Maska mówi, które rekordy modyfikujemy.
- np.nan to standardowa reprezentacja braku dla wielu kolumn numerycznych.

Kiedy używać: Gdy wykryto wartość poza dozwoloną skalą, a poprawna liczba nie jest znana.

Na co uważać: Zamiana na brak to decyzja analityczna. Zapisz liczbę zmian, a następnie uwzględnij nowe braki w procesie imputacji.

11. Flagi braków przed imputacją

Wypełnienie pustego pola usuwa informację, że oryginalnie go brakowało. Flaga zachowuje ten fakt.

```
robocze["brak_oceny"] = robocze["ocena_obslugi"].isna().astype(int)
print(robocze["brak_oceny"].sum())
```

- isna() identyfikuje brakujące wartości.
- astype(int) zamienia True na 1, a False na 0.
- Suma flag mówi, ile rekordów pierwotnie nie miało oceny.

Kiedy używać: Przed uzupełnianiem braków, kiedy sam brak może być informacyjny.

Na co uważać: Flaga musi powstać przed fillna(). Później wszystkie dotychczasowe braki przestaną być widoczne w tej kolumnie.

12. Imputacja globalna: median() i fillna()

Imputacja zastępuje brakującą wartość oszacowaniem. Mediana jest często odporniejsza niż średnia na pojedyncze duże obserwacje.

```
mediana_kwoty = robocze["wartosc_zamowienia"].median()
robocze["wartosc_zamowienia"] = (robocze["wartosc_zamowienia"]
                                 .fillna(mediana_kwoty))
```

- median() wylicza medianę z dostępnych wartości.
- fillna(wartość) zwraca kolumnę z uzupełnionymi brakami.
- Przypisanie wyniku z powrotem do kolumny utrwala zmianę.

Kiedy używać: Gdy chcesz przygotować kompletne pole liczbowe, a dostępnych jest dość obserwacji.

Na co uważać: Imputacja zmienia rozkład danych i może zaniżać zmienność. W modelowaniu ucz medianę na zbiorze treningowym i stosuj ją do walidacyjnego/testowego, aby nie przenosić informacji między zbiorami.

13. Imputacja w grupach: groupby().transform("median")

W różnych segmentach typowe wartości mogą być inne. Można więc obliczać medianę wewnątrz każdej grupy.

```
mediany = (robocze.groupby("rodzaj_dostawy")
           ["dni_dostawy"].transform("median"))
robocze["dni_dostawy"] = robocze["dni_dostawy"].fillna(mediany)
```

- groupby() dzieli wiersze według rodzaju dostawy.
- transform("median") zwraca Series o tej samej długości i indeksach co dane wejściowe.
- Dzięki temu każdy brak może być uzupełniony medianą swojej grupy.

Kiedy używać: Kiedy wartości typowe wyraźnie różnią się między segmentami.

Na co uważać: Jeżeli cała grupa ma braki albo jej nazwa jest pusta, mediana grupowa może nie istnieć. Zaplanuj uzupełnienie rezerwowe, np. medianą globalną.

14. Imputacja kategorii i rzutowanie na liczby całkowite

Dla kategorii brak czasem warto oznaczyć osobną wartością. W liczbach po imputacji można ograniczyć liczbę miejsc i typ.

```
robocze["kanal"] = robocze["kanal"].fillna("nieustalony")
robocze["ocena_obslugi"] = (robocze["ocena_obslugi"]
                            .round().astype(int))
```

- fillna("nieustalony") tworzy jawną kategorię dla nieznanego kanału.
- round() zaokrągla wartości, a astype(int) zmienia typ na całkowity.
- Dla skali ocen całkowity typ może być naturalny, ale informacja o dokładności zostaje utracona.

Kiedy używać: Gdy przygotowujesz ujednolicone kategorie i cechy o zdefiniowanym typie.

Na co uważać: astype(int) nie zadziała przy nieuzupełnionych NaN w zwykłej kolumnie numerycznej. Użyj typu Int64, jeśli chcesz zachować braki całkowite.

15. Tworzenie cech pochodnych

Nowe cechy mogą łączyć lub przeliczać istniejące informacje. Ich sens biznesowy powinien być zrozumiały.

```
robocze["czas_dostawy_tyg"] = (robocze["dni_dostawy"] / 7).round(2)
robocze["laczone_kontakty"] = (robocze["telefony_do_obslugi"]
                               + robocze["wiadomosci_do_obslugi"])
```

```
robocze["koszt_dostawy_proc"] = (
    100 * robocze["koszt_dostawy"] / robocze["wartosc_zamowienia"]
).round(2)
```

- Działania na Series są wykonywane dla wszystkich wierszy.
- Cechy pochodne warto nazwać tak, aby jednostka i znaczenie były jasne.
- Udział procentowy jest ilorazem pomnożonym przez 100.

Kiedy używać: Gdy nowe wskaźniki lepiej opisują zachowanie, czas lub koszty niż pojedyncze pola.

Na co uważać: Przed dzieleniem sprawdź, czy mianownik może wynosić 0. Nie twórz cech ze zdarzeń mających miejsce po prognozowanym zdarzeniu.

16. Flagi warunkowe: np.where() i kodowanie przez map()

NumPy pozwala utworzyć kategorię na podstawie warunku, a słownik zamienić etykiety w liczby.

```
robocze["dostawa_szybka"] = np.where(
    robocze["dni_dostawy"] <= 2, "tak", "nie")
robocze["zwrot_bin"] = robocze["zwrot"].map({"nie": 0, "tak": 1})
```

- np.where(warunek, gdy_prawda, gdy_falsz) działa elementowo.
- map({"nie": 0, "tak": 1}) koduje dwie kategorie liczbowo.
- Wynik np.where może mieć typ tekstowy lub liczbowy zależnie od argumentów.

Kiedy używać: Przy tworzeniu cech binarnych i przygotowaniu etykiety celu do dalszej analizy.

Na co uważać: Jeśli zwrot zawiera inną kategorię niż „tak” i „nie”, map() utworzy brak. Sprawdź wynik przed astype(int).

17. Porównanie stanu przed i po czyszczeniu

Raport braków z dwóch wersji zbioru pozwala pokazać, czy rzeczywiście rozwiązano rozpoznane problemy.

```
braki_przed = surowe.isna().sum()
braki_po = robocze.isna().sum()
porownanie = pd.DataFrame({"przed": braki_przed, "po": braki_po})
porownanie = porownanie.fillna(0).astype(int)
print(porownanie[(porownanie["przed"].gt(0)) | (porownanie["po"].gt(0))])
```

- Series są dopasowywane po nazwach indeksów, tu nazwach kolumn.
- Nowe kolumny mogą nie istnieć w danych surowych, dlatego po zestawieniu pojawia się NaN.
- gt(0) to metoda „większe niż 0”, a | łączy dwa warunki.

Kiedy używać: Po transformacjach, jako dowód tego, jakie braki wystąpiły i co z nimi zrobiono.

Na co uważać: Spadek liczby braków nie dowodzi sam w sobie poprawy jakości. Błędne imputacje również mogą wyzerować licznik braków.

18. Testy jakości: assert, all(), is_unique, set()

Test automatyczny zatrzymuje kod, jeżeli wynik nie spełnia ustalonego warunku.

```
assert robocze["zamowienie_id"].is_unique
assert robocze["ocena_obslugi"].between(1, 5).all()
```

```
assert (robocze["wartosc_zamowienia"] >= 0).all()
assert set(robocze["zwrot"].dropna().unique()) <= {"tak", "nie"}
```

- assert warunek wymaga, aby wyrażenie dało True.
- is_unique sprawdza, czy wartości Series się nie powtarzają.
- all() wymaga, by wszystkie wartości logiczne były prawdziwe.
- set(...) <= zbiór sprawdza, czy zaobserwowane kategorie należą do zbioru dozwolonych.

Kiedy używać: Po każdym większym etapie czyszczenia, żeby szybko wykryć regresję.

Na co uważać: Test zakresu nie zastąpi testu braków. Dla produkcyjnych kontroli lepiej używać jawnych wyjątków lub narzędzi walidacyjnych: assert może być wyłączone przy uruchomieniu Pythona z -O.

19. Sprawdzenie kompletności i liczebności kategorii

Po czyszczeniu nie wystarczy sprawdzić kilku rekordów. Warto ocenić całą kolumnę i zbiór kategorii.

```
print(robocze["kanal"].value_counts(dropna=False))
print(robocze["rodzaj_dostawy"].nunique())
print(robocze.drop(columns=["data_zakonczenia_reklamacji"])
             .isna().sum().sum())
```

- value_counts() pokazuje liczebność każdego kanału.
- nunique() podaje liczbę różnych rodzajów dostawy.
- drop(columns=[...]) wyklucza kolumnę z kontroli, a podwójne sum() podaje łączną liczbę braków w pozostałych polach.

Kiedy używać: Gdy chcesz stwierdzić, czy istotne pola są kompletne po imputacji.

Na co uważać: Nie wykluczaj kolumny z testu bez uzasadnienia. Czasem brak w dacie zdarzenia jest poprawny, jeśli zdarzenie nie wystąpiło.

20. Audyt cech: wykluczenia i wyrażenie listowe

Nie wszystkie dostępne kolumny powinny trafić do modelu. Identyfikatory, informacje z przyszłości i pola audytowe wymagają osobnej decyzji.

```
wyklucz = ["zamowienie_id", "data_zakupu", "data_wyplaty_zwrotu"]
cel = "zwrot_bin"
kandydaci = [kol for kol in robocze.columns
             if kol not in wyklucz + [cel]]
assert "data_wyplaty_zwrotu" not in kandydaci
print(kandydaci)
```

- Wyrażenie listowe buduje nową listę na podstawie nazw kolumn.
- not in sprawdza, czy kolumna nie znajduje się na liście wykluczeń.
- assert upewnia, że wybrane ryzykowne pole nie trafiło do kandydatów.

Kiedy używać: Przed dalszym etapem modelowania i udostępnieniem tabeli do analizy.

Na co uważać: Pola demograficzne, geograficzne lub dochodowe wymagają oceny uzasadnienia, legalności i ryzyka dyskryminacji; samo ukrycie ich na liście nie rozwiązuje problemu.

21. Raportowanie procesu na wykresie

Wizualizacja stanu początkowego i efektów pozwala szybko ocenić skalę zmian.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
miary = pd.Series({"rekordy wejściowe": len(surowe),
                   "rekordy końcowe": len(robocze),
                   "kopie": surowe.duplicated().sum()})
miary.plot(kind="bar", ax=ax)
ax.set_title("Kontrola przygotowania danych")
ax.set_ylabel("Liczba")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```

- plt.subplots() tworzy figurę i obszar rysunku (ax).
- pd.Series ze słownika jest prostą strukturą danych do wykresu.
- rotation i ha obracają oraz wyrównują podpisy osi.

Kiedy używać: Do prezentowania liczby usuniętych rekordów i skali rozpoznanych problemów.

Na co uważać: Na wykresie o jednej osi nie porównuj bez komentarza wielkości o zupełnie różnych znaczeniach, np. liczby rekordów i procentów.

22. Eksport CSV i ponowny odczyt

Gotowy zbiór należy zapisać i sprawdzić, czy z dysku daje się go odczytać.

```
from pathlib import Path

plik = Path("zamowienia_po_czyszczeniu.csv")
robocze.to_csv(plik, index=False, encoding="utf-8-sig",
               date_format="%Y-%m-%d")
sprawdzenie = pd.read_csv(plik)
print(sprawdzenie.shape)
```

- to_csv() zapisuje dane do pliku tekstowego.
- index=False zapobiega dopisywaniu technicznego indeksu DataFrame.
- utf-8-sig bywa praktyczne przy otwieraniu CSV z polskimi znakami w Excelu.
- date_format kontroluje zapis dat; read_csv() potwierdza możliwość odczytu.

Kiedy używać: Na końcu procesu, kiedy dane mają trafić do kolejnego etapu.

Na co uważać: Zapis CSV nie zachowuje wszystkich typów pandas, np. daty po ponownym read_csv mogą być tekstem. Sprawdź nie tylko shape, ale też typy i najważniejsze kategorie.

23. Ograniczenia uproszczonego czyszczenia

Jedna reguła nie zawsze pasuje do każdego projektu. Decyzje trzeba sprawdzić w kontekście źródła i celu.

```
globalna_mediana = robocze["wartosc_zamowienia"].median()
print(f"Mediana w pełnym zbiorze: {globalna_mediana:.2f}")
```

- Gdy przygotowujesz zbiór do przyszłego modelowania, parametry imputacji i kodowania dopasowuj na części treningowej.
- Sprawdź, czy mediany grupowe mają sens przy małych grupach i czy każda grupa posiada obserwacje.
- Kontroluj wartości po imputacji, liczbę braków oraz konsekwencje biznesowe każdej naprawy.
- Nie twórz zmiennych z informacji powstałych po momencie prognozy.

Kiedy używać: Przy przejściu z ćwiczenia o jakości danych do pełnego projektu modelowania.

Na co uważać: Czysta tabela nie oznacza automatycznie danych bezstronnych, reprezentatywnych ani gotowych do użycia w decyzjach.

24. Proces od surowych danych do gotowego pliku

Etap	Operacja	Dowód wykonania
1. Odczyt	read_csv(), copy()	Rozmiar i typy wejścia
2. Powtórzenia	duplicated(), drop_duplicates()	Liczba usuniętych kopii
3. Teksty	strip(), lower(), map()	Zestaw kategorii po zmianie
4. Daty i reguły	to_datetime(), between(), loc	Liczba błędnych wartości i dat
5. Braki	isna(), flagi, fillna()	Zmiana braków przed i po
6. Cechy	operacje na Series, np.where()	Definicje nowych zmiennych
7. Testy	assert, is_unique, all()	Wyniki kontroli jakości
8. Audyt i eksport	lista wykluczeń, to_csv()	Lista cech i poprawny odczyt pliku

26. Kompletny przykład czyszczenia małego zbioru

Poniższy kod można uruchomić w samodzielnym notebooku bez przygotowywania pliku wejściowego. Zbiór zawiera jeden powtórzony rekord, nieprawidłową ocenę, brak kwoty oraz warianty zapisu kanału. Proces celowo kończy się kontrolą wyniku.

```
import pandas as pd
import numpy as np

surowe_demo = pd.DataFrame({
    "transakcja_id": [201, 202, 203, 203, 204],
    "kanal": [" Web ", "SALON", "web", "web", None],
    "ocena": [5, 7, None, None, 4],
    "kwota": [120.0, None, 60.0, 60.0, 80.0]
})

def czysc_demo(dane):
    out = dane.drop_duplicates().copy()
    out["kanal"] = (out["kanal"].astype("string").str.strip()
                    .str.lower().map({"web": "online", "salon": "stacjonarny"}))
    bledna = out["ocena"].notna() & ~out["ocena"].between(1, 5)
    out.loc[bledna, "ocena"] = np.nan
    out["brak_oceny"] = out["ocena"].isna().astype(int)
    out["ocena"] = out["ocena"].fillna(out["ocena"].median()).round().astype(int)
    out["kwota"] = out["kwota"].fillna(out["kwota"].median())
    out["kanal"] = out["kanal"].fillna("nieznany")
    return out

czyste_demo = czysc_demo(surowe_demo)
assert len(czyste_demo) == 4
assert czyste_demo["transakcja_id"].is_unique
assert czyste_demo["ocena"].between(1, 5).all()
assert czyste_demo.isna().sum().sum() == 0
print(czyste_demo)
```

Jak interpretować rezultat? Po deduplikacji pozostają 4 transakcje. Ocena 7 jest oznaczona jako brak przed uzupełnieniem; rekord z identyfikatorem 203 już wcześniej nie miał oceny. Flaga brak_oceny przyjmuje wartość 1 w dwóch wierszach. Mediana dostępnych poprawnych ocen 4 i 5 wynosi 4.5, a po zaokrągleniu otrzymujemy wartości całkowite. Brak kwoty uzupełnia mediana pozostałych kwot. Brakujący kanał zostaje kategorią „nieznany”.

Co pokazują testy? Nie ma powtórzonych identyfikatorów, wszystkie oceny mieszczą się w dozwolonym zakresie, a żadna z kolumn nie zawiera braków. To nie dowodzi, że wszystkie oszacowania są idealne; potwierdza tylko spełnienie wskazanych reguł.

27. Różnica między agregacją a transformacją

To częsta przyczyna nieporozumień przy imputacji grupowej. Agregacja groupby(...).median() zwraca jedną wartość dla każdej grupy. transform("median") zwraca wartość odpowiadającą każdemu oryginalnemu wierszowi, dzięki czemu można wykorzystać ją w fillna().

```
przyklad = pd.DataFrame({
    "typ": ["A", "A", "B", "B"],
    "czas": [2.0, None, 8.0, 10.0]
})
print(przyklad.groupby("typ")["czas"].median())
print(przyklad.groupby("typ")["czas"].transform("median"))
```

Pierwszy wynik ma dwa wiersze: A i B. Drugi ma cztery wartości: 2, 2, 9 i 9, dopasowane do obserwacji. Tylko ten drugi można bezpośrednio wykorzystać jako „osobną medianę" dla każdego wiersza.

28. Co może pójść nie tak podczas czyszczenia?

Operacja	Nieoczekiwany efekt	Kontrola i reakcja
map(słownik)	Nieznana kategoria zmienia się w NaN.	Porównaj unique() przed i po; uzupełnij słownik lub oznacz wyjątek.
to_datetime(errors="coerce")	Błędna data staje się NaT.	Policz nowe braki i sprawdź wzorzec formatu.
fillna(medianą)	Rozkład ma sztuczne skupienie przy medianie.	Zachowaj flagi braków; oceń liczbę imputacji.
groupby().transform()	Grupa bez danych nie ma mediany.	Dodaj regułę zapasową albo osobny raport.
astype(int)	Nie zadziała dla pozostałego NaN.	Sprawdź isna(); rozważ typ Int64.
dzielenie kolumn	Mianownik 0 tworzy niepożądane wartości.	Kontroluj mianownik i reguły biznesowe.
drop_duplicates()	Może usunąć prawidłowe powtórzenia zdarzeń.	Ustal jednostkę obserwacji i identyfikator.
to_csv(), read_csv()	Zmienia się reprezentacja dat lub typów.	Zweryfikuj dtypes, liczbę wierszy i wartości po odczycie.

29. Jak ocenić, czy proces jest gotowy do powtórzenia?

- Przyjmuj surowe dane jako argument funkcji, a przed zmianą ustal dozwolone wartości i powód korekty.
- Twórz flagi i liczniki zmian przed imputacją, a decyzje dokumentuj w dzienniku transformacji.
- Po etapach sprawdzaj rozmiar, braki, kategorie i zakresy. Uruchamiaj testy, kontroluj wyciek informacji, a po eksporcie zweryfikuj ponowny odczyt.